Value Screener

Requirements and design for a lean value-vs-price stock picker · v1.0 · 4 November 2026

In one sentence: a Python tool that takes your portfolio, an ETF or a list of tickers, values each company only from its latest filing re-priced to today's metal prices, and tells you which are cheap (buy zone), fair, or too expensive (sell zone) – with buy and sell price levels, option strike levels, and a warning whenever the data is out of date.

1. Goals and non-goals

Goals	Non-goals (for now)
Value any miner, royalty company, developer, physical trust, ETF, tech or energy stock with the same methods as the Value vs Price reports.	No automatic trading or order placement.
Rank by margin of safety and flag buy / sell zones and option strike levels.	No analyst targets or forecasts as inputs.
Show what metal price or growth rate the share price already assumes.	No full automation of filings parsing in v1 (company files are updated by hand or with Claude).
Every number traceable to a filing, with a date; stale data flagged, never silently used.	No web dashboard until Stage 4.
Lean: a Python package and command line, runs locally in seconds, offline if needed.	No database in v1 – plain YAML and CSV files.

2. Users and use cases

#	Use case	Trigger	What you get
U1	Weekly scan	Sunday evening	Ranked list of the watchlist: buy zone, watch, hold, sell zone
U2	Earnings refresh	A company reports	Update its YAML file; re-run; see how value moved
U3	Before an options trade	Selling a put or call	Put strike at or below 12x value; call strike near 25x value
U4	Check an ETF	Considering URNM, GDX, SIL	Look-through value of the top holdings vs ETF price
U5	Portfolio review	Monthly	Your own holdings ranked, with sell-zone warnings
U6	What-if	Metal price moves	Values at −25% ... +30% metals, or another spot price

3. Functional requirements

Priority: **M** = must, **S** = should, **C** = could. Stage = when it is built (see roadmap).

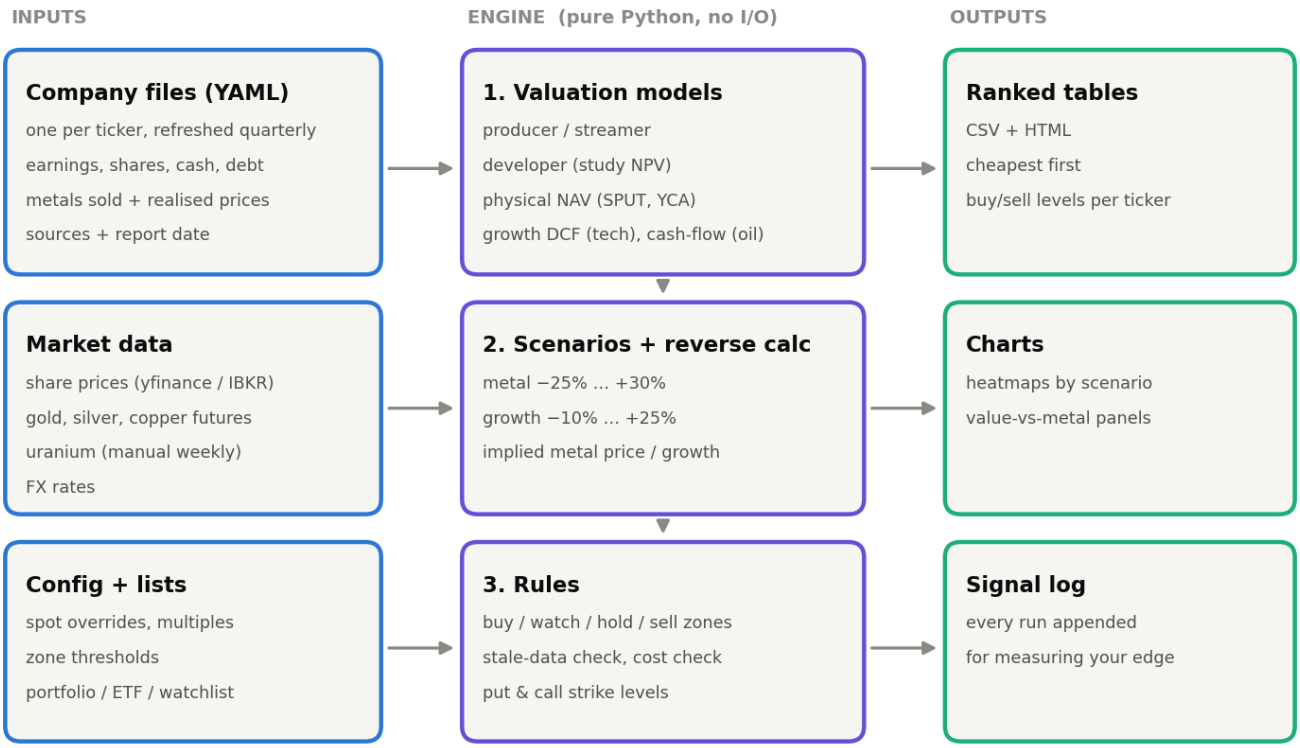
ID	Requirement	Pri	Stage
FR-1	Load one YAML file per company; validate required fields and units; reject files with missing source or report date.	M	1
FR-2	Accept input as: (a) a ticker list on the command line, (b) a watchlist file, (c) a portfolio file, (d) an ETF holdings file.	M	1
FR-3	Producer model: owner earnings = adjusted net income + add-backs + D&A; − sustaining capex, annualised, per share.	M	1
FR-4	Re-price each metal sold from the quarter's realised price to today's spot, after royalties and tax.	M	1
FR-5	Value = owner earnings × multiple + net cash per share, at 12x / 15x / 20x / 25x (configurable; Chevron-type 10x/13x).	M	1
FR-6	Developer model: company study NPV interpolated to spot (linear between published price cases), × ownership, + cash − debt.	M	1
FR-7	Physical model (SPUT, Yellow Cake): NAV = holdings × spot + other net assets; or scale a published NAV to spot.	M	1
FR-8	Growth model (tech): 5-year DCF at 10% with 15x exit; report no-growth value and the growth the price implies.	M	1
FR-9	Reverse calculation: metal price (producers) or growth rate (tech) that makes value equal to price.	M	1
FR-10	Scenario grid: values at metals −25%, −10%, spot, +15%, +30% (and growth −10% … +25% for tech).	M	1
FR-11	Zones: BUY (margin of safety ≥ 20% at 12x and cost check passes), WATCH (0–20%), HOLD, SELL (price ≥ 25x value or ≥ 1.5x NAV).	M	1
FR-12	Stale check: flag STALE if period end > 100 days ago or the next report date has passed; stale rows never get a BUY.	M	1
FR-13	Cost check for miners: all-in sustaining cost ≤ 60% of spot metal price (configurable).	S	1
FR-14	Option levels: suggested put strike = 12x value; covered-call strike = 25x value.	S	1
FR-15	ETF look-through: weighted value ÷ price of the valued holdings; report coverage %.	S	1
FR-16	Outputs: ranked CSV, self-contained HTML report, scenario heatmap PNGs; console summary.	M	1
FR-17	Live prices from yfinance (or IBKR); fall back to the price stored in the YAML, marked with its date.	M	1
FR-18	Append every run's zones and levels to a signal log (CSV) for measuring your edge.	S	1
FR-19	Technical signals: RSI(14) daily and weekly, MACD(12,26,9), 200-day SMA – combined with zones into a trigger.	M	2
FR-20	Scale-in plan when value says buy but the trend is down: thirds on RSI oversold / reclaim of the 200-day.	S	2
FR-21	Quarterly refresh helper: paste a results release → draft YAML for review (Claude-assisted).	C	3
FR-22	Spring Boot REST API exposing runs, companies and signals; scheduled weekly job; dashboard; AWS deploy.	C	4

4. Non-functional requirements

ID	Requirement
NFR-1 Accuracy	Stage 1 must reproduce the published Value vs Price reports to within 1% for every company (acceptance test).
NFR-2 Traceability	Every input carries its source and date; every output row shows the results period and price date.
NFR-3 No silent guessing	Assumptions live in the YAML under 'assumptions' and are printed in the HTML next to the company.
NFR-4 Speed	100 tickers in under 30 seconds with live prices; under 2 seconds offline.
NFR-5 Offline	Runs with no network using stored prices and the spot values in config.
NFR-6 Testable	Engine is pure functions (no I/O); unit tests for each model and the rules; pytest runs in CI.
NFR-7 Simple	Python 3.10+, dependencies: pyyaml, pandas, numpy, scipy, matplotlib; yfinance optional.
NFR-8 Safe	No broker credentials in files; IBKR access (Stage 2+) via environment variables only.

5. Architecture

Three layers. Inputs are plain files and market feeds; the engine is pure Python with no I/O, so it is easy to test and later to wrap in a Java/Spring Boot service; outputs are files you can open or post.



Package layout

```
value-screener/
  valuescreen/
    models.py      valuation models (producer, developer, physical, growth, look-through)
    rules.py       zones, stale check, cost check, option levels
    scenarios.py   metal and growth scenario grids, reverse calculations
    loader.py      YAML loading + validation, fx conversion
    prices.py      live prices (yfinance) with offline fallback
    report.py      CSV, HTML and heatmap outputs, signal log
    cli.py         command line:  python -m valuescreen run --list watchlist
  data/companies/*.yaml   one file per company (filled for all ~40 companies valued so far)
  data/etfs/*.yaml        ETF holdings and weights
  data/lists/*.yaml       watchlists and your portfolio
  config.yaml             spot prices, FX, multiples, zone thresholds
  tests/                  acceptance tests against the published reports
  output/                 generated reports (git-ignored)
```

6. Data model – company file

Amounts are in millions of the reporting currency for the period stated; per-share and per-unit amounts in units. Ounces and pounds are in millions (Moz, Mlb) per period.

```
ticker: AEM                     # display ticker;  yahoo: AEM  (symbol for live price)
name: Agnico Eagle Mines
sector: gold                    # gold | silver | uranium | copper | royalty | tech | energy
model: producer                 # producer | developer | physical | growth | cashflow
currency: USD                   # reporting currency of the inputs below
period: {label: Q3-2026, end: 2026-09-30, reported: 2026-10-28, per_year: 4, next_report: 2027-02-12}
earnings:                       # $m for the period
  adj_net_income: 1324.349
  addbacks: 0                   # e.g. non-cash inventory charges
  da: 457.447                   # depreciation & amortisation
  sustaining_capex: 296.173
shares_m: 505.546
balance: {cash: 3613.0, debt: 197.0}          # or net_cash: 3416.023
metals:                         # sold in the period, for re-pricing to spot
  - {metal: gold, units: 0.825084, realised: 4264, aisc: 1648}
tax_rate: 0.35
royalty_rate: 0.0               # share of a price change lost to price-linked royalties
multiples: [12, 15]             # optional override of the default ladder
price: {value: 167.21, currency: USD, date: 2026-11-03}
sources: ["Q3-2026 results, SEC Ex 99.1, 28 Oct 2026"]
assumptions: []
```

Model-specific blocks: **developer** → npv_curve (price points and NPVs from the company study), ownership; **physical** → holdings (Mlb or Moz) and other_net_assets, or a published nav_ref at a reference spot; **growth** → net income or EPS per period, investment gains to strip, ADR ratio; **cashflow** → operating cash ex working capital and capex, optional normalised block (e.g. Chevron on 2025 oil).

7. Valuation formulas

Model	Formula	Used for
Producer	OE_{rep} = (adj NI + add-backs + D&A; − sustaining capex) × periods/yr ÷ shares $OE_{spot} = OE_{rep} + \Sigma_{metals}$ units × periods × (spot − realised) × (1 − royalty) × (1 − tax) ÷ shares Value(m) = OE_{spot} × m + net cash ÷ shares	Gold, silver, uranium, copper miners; streamers and royalty companies (D&A; and capex = 0)
Developer	NAV = NPV(spot) × ownership + cash − debt, ÷ shares NPV(spot) interpolated linearly between the study's own price cases	AbraSilver, NexGen, Denison
Physical	NAV = holdings × spot + other net assets, ÷ units; or NAV_{ref} × (holdings × spot + other) ÷ (holdings × $spot_{ref}$ + other)	Sprott Physical Uranium, Yellow Cake
Growth DCF	Value(g) = OE × [$\Sigma_{t=1..5}$ $((1+g)/1.10)^t$ + 15 × $((1+g)/1.10)^5$] + net cash No-growth value = Value(0) ≈ 13.1 × OE + net cash; implied g solves Value(g) = price	Tech, semis
Cash-flow	OE = (operating cash flow ex working capital − capex) × periods ÷ shares; multiples 10x/13x	Oil majors
ETF look-through	Coverage ratio = Σ $weight_i$ × ($value_i$ ÷ $price_i$) ÷ Σ $weight_i$ over valued holdings	URNM, GDX, SIL
Reverse – metal	spot* = spot + ((price − net cash)/m − OE_{spot}) ÷ sensitivity, sensitivity = Σ units × periods × (1 − royalty)(1 − tax) ÷ shares	What metal price the share price assumes

Margin of safety = (value at the cautious multiple − price) ÷ value at the cautious multiple. Negative = paying for things to get better.

8. Rules and zones

Zone	Producers / streamers	Developers & physical	Growth (tech)
BUY	Margin of safety ≥ 20% at 12x, cost check passes, data not stale	Price ≤ 0.8 × NAV (physical ≤ 0.85 × NAV)	Implied growth ≤ 5%/yr
WATCH	Margin of safety 0–20% at 12x	0.8–1.0 × NAV	5–10%/yr
HOLD	Price between 12x and 25x value	1.0–1.5 × NAV	10–25%/yr
SELL / write calls	Price ≥ 25x value	Price ≥ 1.5 × NAV (physical ≥ 1.1 × NAV)	Implied growth ≥ 25%/yr
STALE	Period end > 100 days or next report date passed – shown, never BUY	Same	Same

All thresholds live in config.yaml. Option levels: put strike = value at 12x (or 0.8 × NAV); call strike = value at 25x (or 1.5 × NAV). Stage 2 adds your technical trigger: a BUY becomes 'BUY NOW' only when daily RSI is oversold near support or MACD turns up, and price is above or reclaiming the 200-day average; otherwise 'BUY – scale in'.

9. Outputs

Output	Contents
ranked.csv	ticker, model, period, price & date, owner earnings (reported / spot), net cash, value at each multiple or NAV, margin of safety, implied metal or growth, zone, put & call levels, stale flag, cost ratio
report.html	Self-contained page: ranked table coloured by zone, scenario heatmap, each company's assumptions and sources
heatmap_*.png	Value per scenario versus price, like the PDF reports
signals_log.csv	Every run appended: date, ticker, zone, levels – the basis for measuring your edge after 4–6 quarters

10. Roadmap

Stage	Scope	Effort	Calendar (part-time)
1 – Core (delivered with this document)	FR-1 … FR-18: models, scenarios, zones, ETF look-through, CSV/HTML/PNG, data files for ~40 companies	10–15 h	done
2 – Your trading rules	FR-19, FR-20: RSI / MACD / 200-day from daily prices; trigger + scale-in plan; IBKR prices	6–10 h	+1 week
3 – Refresh & tracking	FR-21: release → YAML helper; signal-log analysis vs GDX / SIL; options-level checks	8–12 h	+1–2 weeks
4 – Productionise	FR-22: Spring Boot API over the engine (or a Java port), scheduled runs, dashboard, AWS deploy	25–40 h	+3–5 weeks

11. Risks and mitigations

Risk	Mitigation
Input errors (wrong units, wrong quarter)	Schema validation; acceptance tests; source + date on every file; units fixed by convention (Moz, Mlb, $m)
Stale numbers used for decisions	STALE flag blocks BUY; next_report date in each file
Cyclical peak earnings look cheap (Micron, Chevron)	Optional normalised block; 'cyclical' flag shown next to the zone
Value traps (jurisdiction, high cost)	Cost check (AISC ÷ spot); notes field printed in the report; human decision stays with you
Price feed breaks	Offline fallback to stored price with its date; prices file cached per run
Over-trusting the model	Signal log + quarterly review against GDX/SIL before sizing positions on it

12. Acceptance criteria – Stage 1

- All company files load and validate; invalid files are reported with the field that failed.
- For every company in the four published reports, value at each multiple / NAV / no-growth value is within 1% of the report.
- Implied gold/silver price and implied growth match the reports to within 1%.
- A STALE company is never placed in the BUY zone.
- Runs offline in under 2 seconds and produces ranked.csv, report.html and heatmap PNGs.
- pytest passes.

Running it: `pip install -r requirements.txt` then `python -m valuescreen run --list watchlist` (or `--tickers AEM,NEM,HL`, `--etf URNM`, `--live` for live prices). Outputs land in `output/`.